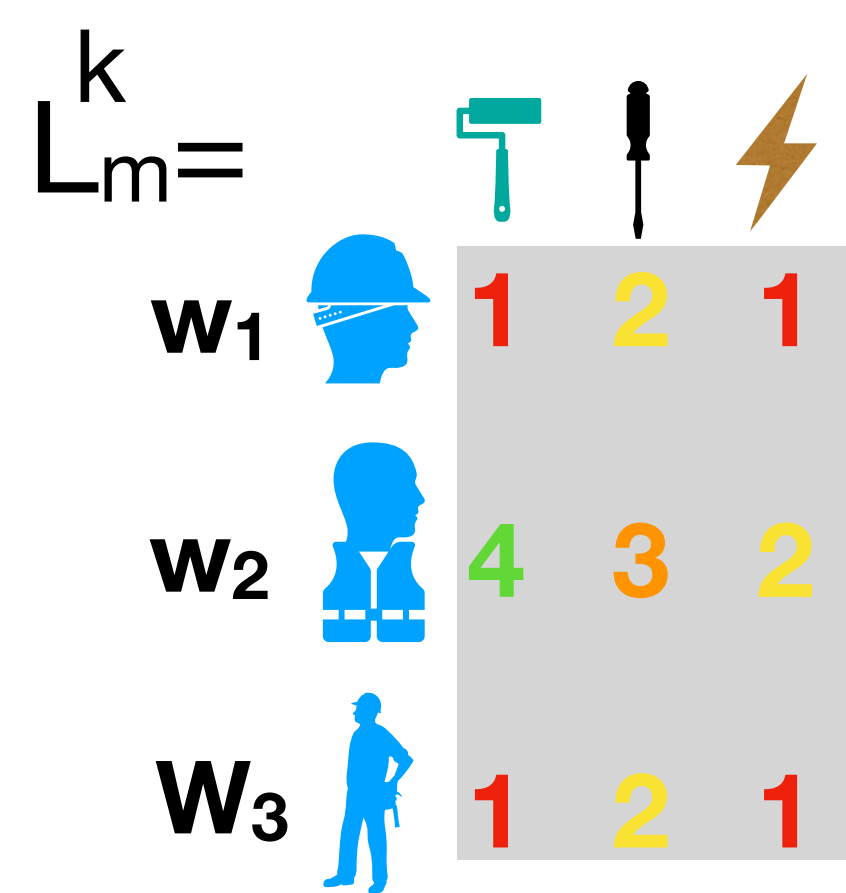
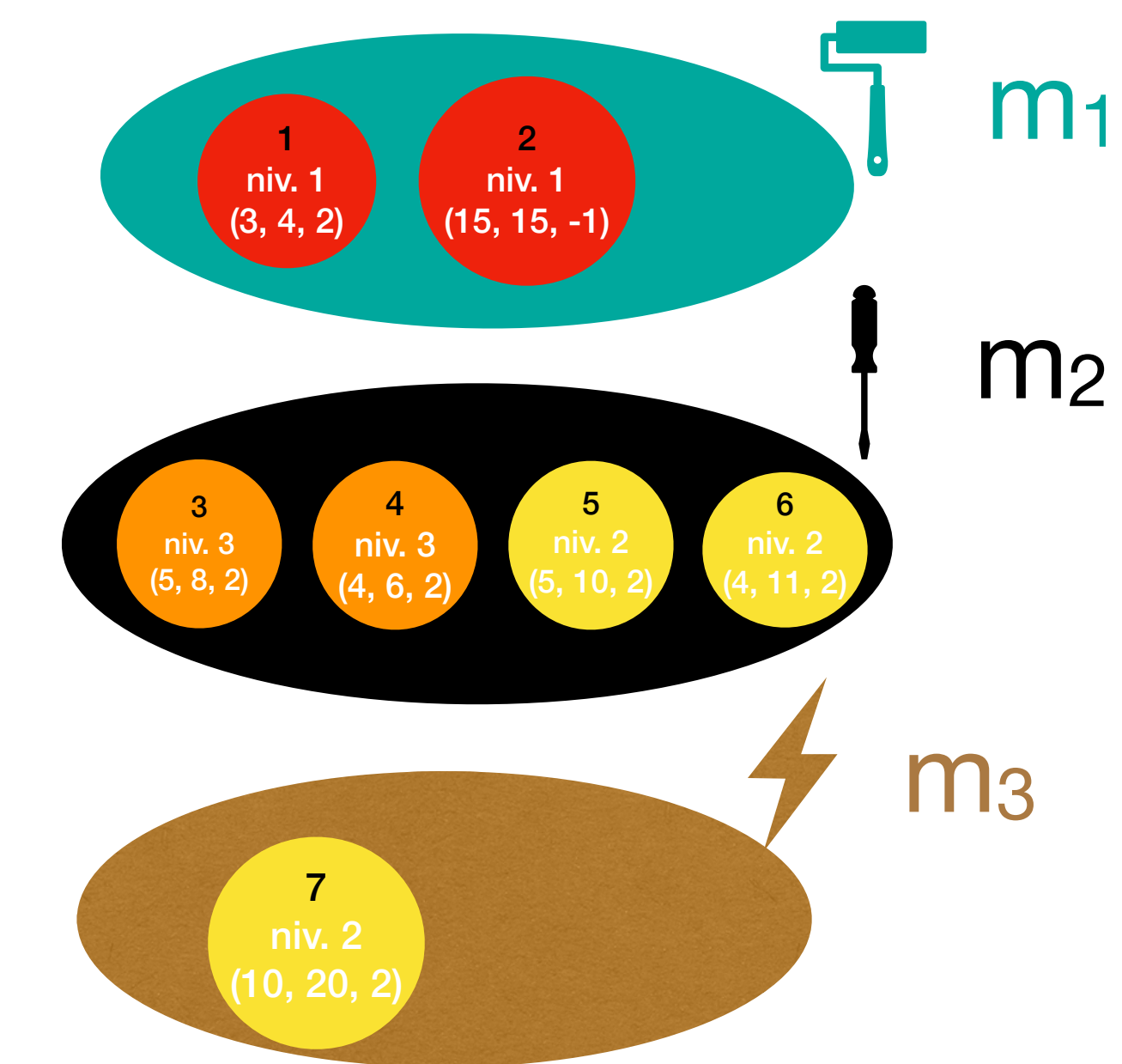
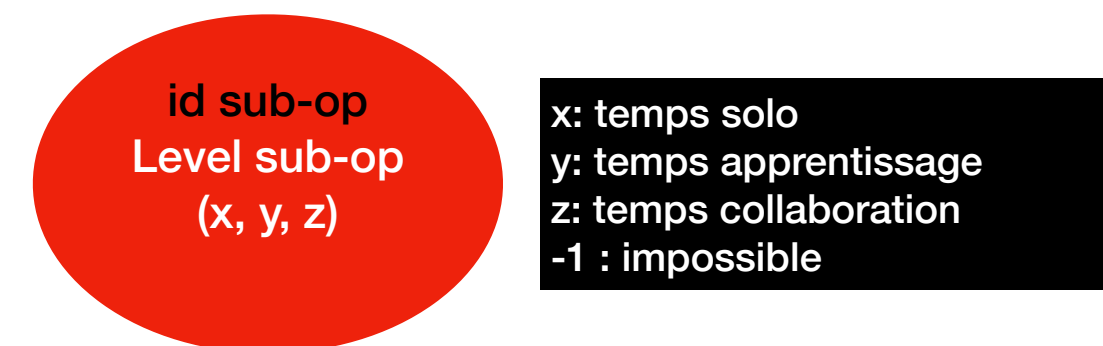


# Structure d'une instance



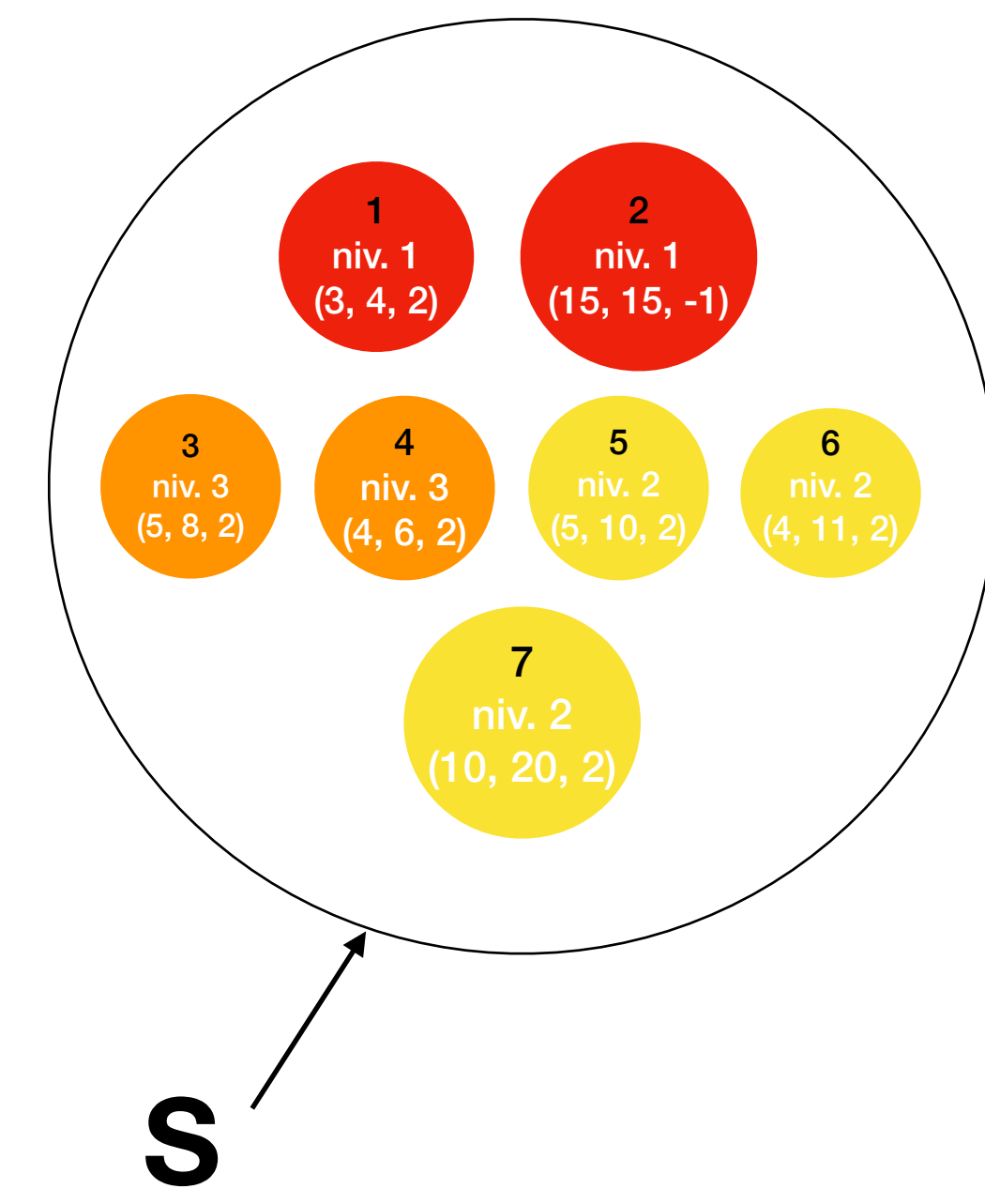
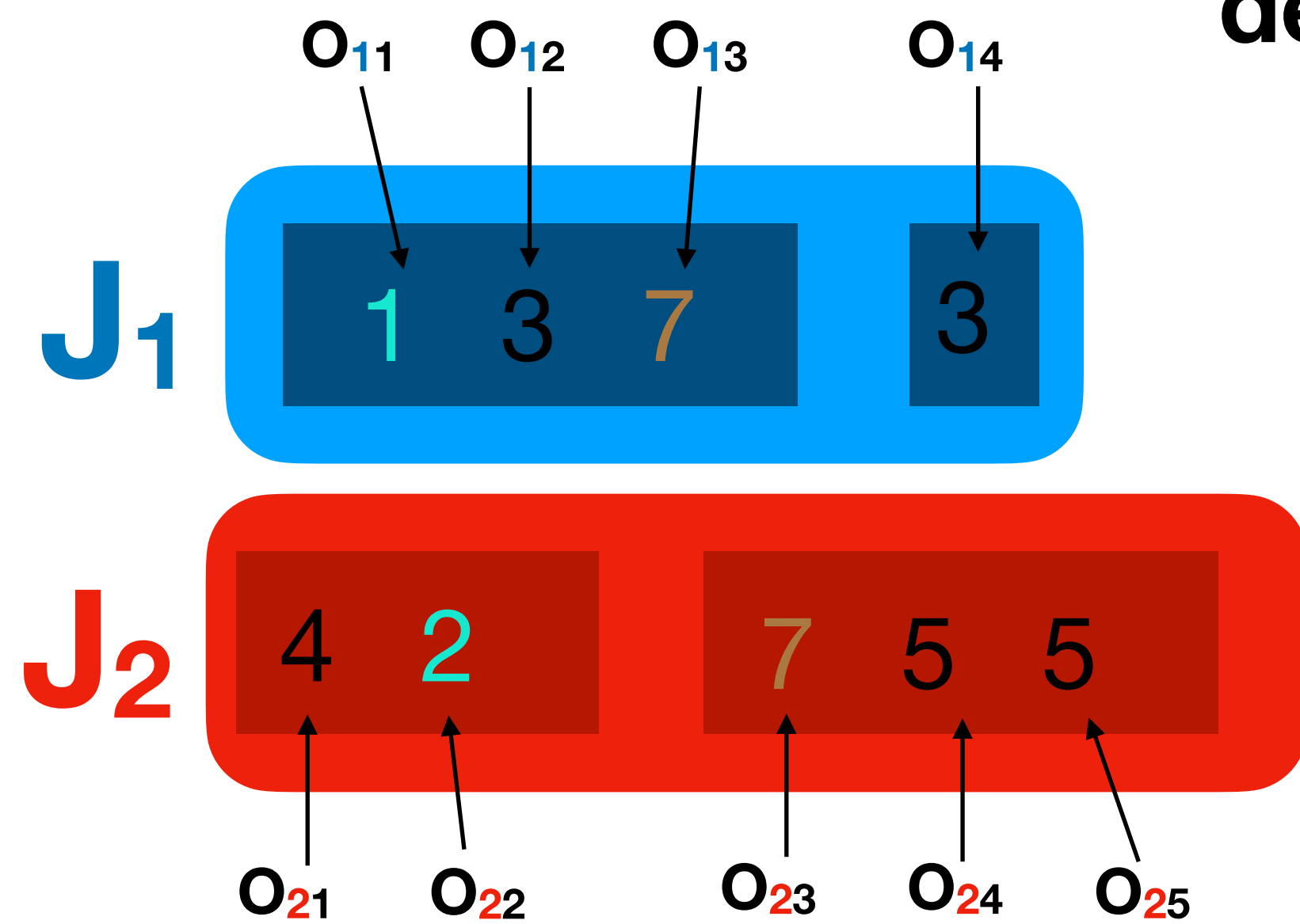
Légende :



## Vocabulaire:

- Une tâche signifie une opération  $O_{ij}$
- $O_{ij} \in \mathbf{S}$

Chaque opération  $O_{ij}$  appartient à l'ensemble  $\mathbf{S}$  des tâches



# Explication de quelques contraintes du modèle

**LMIN** : Level du worker le moins compétent pour la tâche  $O_{ijs}$   
 Utile pour savoir le mode de la tâche (solo, apprentissage ou collab)

$$LMIN_{ijs} = \min_{k \in W} x_{ijsk} \cdot L_m^k$$

## Linéarisation du min

$$LMIN_{ijs} \leq x_{ijsk} \cdot L_m^k + M \cdot (1 - x_{ijsk})$$

$$\forall i \in J, j \in J_i, s \in O_{ij}, k \in W$$

(9)

Plus Petit que le niveau de chaque opérateur sur la tâche

$$LMIN_{ijs} \geq x_{ijsk} \cdot L_m^k - M \cdot (1 - \delta MIN_{ijsk})$$

$$\forall i \in J, j \in J_i, s \in O_{ij}, k \in W$$

(10)

Va fixé  $LMIN_{ijs}$  au niveau de compétence du worker le moins compétent

$$\sum_{k \in W} \delta MIN_{ijsk} = 1$$

$$\forall i \in J, j \in J_i, s \in O_{ij}$$

(11)

Pour une tâche on sélectionne le niveau que d'une seule personne

# Une tâche est exécuté soit en **solo**, en **apprentissage** ou en **collaboration**

$$\forall i \in J, j \in J_i, s \in O_{ij}$$

(6)

$$z_{ijs}^0 + z_{ijs}^1 + z_{ijs}^2 = 1$$

Nombre de personne affecté à la tâche

$$\sum_{k \in W} x_{ijsk} = 1 \cdot z_{ijs}^0 + 2 \cdot z_{ijs}^1 + 2 \cdot z_{ijs}^2$$

Si 1 personne affecté, alors  $z_{ijs}^0 = 1$

Si 2 personnes affecté, alors soit  $z_{ijs}^1 = 1$ , soit  $z_{ijs}^2 = 1$

$$z_{ijs}^1 + z_{ijs}^2 \leq 1$$

Contrainte peut être inutile

$$\forall i \in J, j \in J_i, s \in O_{ij}, r = O_{ijs}$$

(12)

$$LMIN_{ijs} + M \cdot z_{ijs}^1 \geq d_r$$

solo :  $z_{ijs}^1$  non contraint

**appr :  $z_{ijs}^1 = 1$**

collab :  $z_{ijs}^1$  non contraint

$$LMIN_{ijs} - M \cdot z_{ijs}^2 \leq d_r + M \cdot \left( 2 - \sum_{k \in W} x_{ijsk} \right)$$

solo :  $z_{ijs}^2$  non contraint

appr :  $z_{ijs}^2$  non contraint

**collab :  $z_{ijs}^2 = 1$**

# Overlap Entre les taches exécuté par un worker

$$\forall k \in W, i \in J, j \in J_i, h \in J, g \in J_h, s \in O_{ij}, z \in g, (i, j, s) \neq (h, g, z)$$

$$f_{hgzk} \geq d_{hgzk} + t_{r'}^0 \cdot z_{hgz}^0 + t_{r'}^1 \cdot z_{hgz}^1 + t_{r'}^2 \cdot z_{hgz}^2 - M \cdot (1 - x_{hgzk})$$

$$f_{ijsk} \geq d_{ijsk} + t_r^0 \cdot z_{ijs}^0 + t_r^1 \cdot z_{ijs}^1 + t_r^2 \cdot z_{ijs}^2 - M \cdot (1 - x_{ijsk})$$

Si tache pas fait par k, alors  
fait par un autre opérateur  
est  $F_{ijsk}$  = durée de  $O_{ijsk}$ .  
Permet de rendre  $F_{ijsk}$  à -M

$$c1: d_{ijsk} \geq F_{hgzk} - M \cdot \delta_{ijhgszk} \quad \text{avec} \quad F_{ijsk} = d_{ijsk} + t_r^0 \cdot z_{ijs}^0 + t_r^1 \cdot z_{ijs}^1 + t_r^2 \cdot z_{ijs}^2 - M(1 - x_{ijsk})$$

Si k fait  $O_{ijs}$  alors vrai fin  
Sinon  $F_{ijsk} = -M$

$$c2: d_{hgzk} \geq F_{ijsk} - M(1 - \delta_{ijhgszk})$$



# Au moins 1 personnes Ayant un niveau de compétence requis pour l'exécution d'une tâche

W1 :2 niv 3

W2 : 2

$$\forall i \in J, j \in J_i, j' \in J_i, s \in O_{ij}, k \in W, L_m^k < d_r$$

Worker k n'a pas le niveau requis pour la tâche  $O_{ijs}$

- S'il est affecté alors une autre personne doit être affecté avec lui
- Sinon contrainte non actif (big M)

$$x_{ijsk} + \sum_{k' \in W, k' \neq k} x_{ijsk'} \geq M \cdot (x_{ijsk} - 1) + 2$$

- Si k fait la tâche alors la personne avec lui doit forcément avoir le niveau de compétence requis
- Sinon contrainte non actif

$$\sum_{k' \in W, k' \neq k} x_{ijsk'} \cdot L_m^{k'} \geq d_r \cdot x_{ijsk}$$

## Si 2 personnes sont affecté à la même tâche alors elles doivent la commencer en même temps

$$\forall i \in J, j \in J_i, s \in O_{ij} k_1 \in W, k_2 \in W$$

$$d_{ijsk_1} \leq d_{ijsk_2} + M \cdot (2 - x_{ijsk_1} + x_{ijsk_2})$$

$$d_{ijsk_2} \leq d_{ijsk_1} + M \cdot (2 - x_{ijsk_1} + x_{ijsk_2})$$

- Si la tâche est fait à 2 alors les deux contraintes sont actif pour les 2 personnes affecté seulement
- Contrainte inactif sinon

En lisant l'article suivant j'ai repris leurs méthode pour modéliser cette contrainte

Florencia, Jessica, Lorraine Trilling, Thierry Moyaux, et al. « Mixed Integer Linear and Constraint Programming for Dual-Resource Scheduling with Synchronization in Emergency Departments ». Art. de conf. presented sur 11th International Conference on Control, Decision and Information Technologies (CoDIT 2025). 15 juillet 2025. <https://hal.science/hal-05290044>.

# Effet d'apprentissage des taches d'un corps de métier

Un worker k n'ayant pas le niveau requis pour effectuer une tache d'un corps de métier peut l'exécuter accompagné d'une personne ayant le niveau requis. Apres cette exécution le niveau de compétence pour les tache de ce corps de métier de la personne formé augmentera

• Si k n'a pas le niveau de compétence requis

•  $tasks^L$  = nombre de tache fait par k tq il n'avait pas le niveau requis, donc fait en apprentissage

$$\forall k \in W, m \in M, s \in m, L_m^k < d_r,$$

$$tasks^L = \sum_{\substack{i \in J \\ j \in J_i \\ s \in O_{ij}}} x_{ijsk}$$

$$l_m^k \leq L_m^k + tasks^L \cdot coef^L \qquad O\_111 \text{ d-} > 2$$

$$l_m^k \leq L_m^k + 1$$

$$l_m^k \geq L_m^k$$

$$l_m^k \leq 4$$

**Augmentation du niveau de compétence pour worker k sur corps de métier m**

## Savoir si un worker a le niveau pour $O_{ijs}$

$$\forall i \in J, j \in J_i, s \in O_{ij}, k \in W, r = O_{ijs}, r \in m$$

$$L_m^k + M \cdot (1 - has\_level_{ijsk}) \geq d_r$$

$$L_m^k - M \cdot has\_level_{ijsk} \leq d_r - \epsilon$$

$$\epsilon > 0$$

- $\epsilon$  car si niveau de compétence égale au niveau de difficulté de la tâche, le Worker k ne sera pas considéré comme ayant le niveau requis

## Savoir si un worker a fait la tâche en tant que tuteur

$$\forall i \in J, j \in J_i, s \in O_{ij}, k \in W, r = O_{ijs}, r \in m$$

$$is\_tutor_{ijsk} \leq x_{ijsk}$$

$$is\_tutor_{ijsk} \leq z_{ijs}^1$$

$$is\_tutor_{ijsk} \leq has\_level_{ijsk}$$

$$is\_tutor_{ijsk} \geq x_{ijsk} + z_{ijs}^1 + has\_level_{ijsk} - 2$$

$$tab\_count\_tasks\_has\_tutor_k^m \quad + = \quad is\_tutor_{ijsk}$$

### Linéarisation du ET LOGIQUE

$$x_{ijsk} \cdot z_{ijs}^1 \cdot has\_level_{ijsk}$$

tab\_count [k][m] = nombre de fois où l'opérateur k a été tuteur pour une tâche de corps de métier m

# Savoir si un worker a fait la tache en tant qu'apprenti

$$\forall i \in J, j \in J_i, s \in O_{ij}, k \in W, r = O_{ijs}, r \in m$$

$$is\_apprenti_{ijsk} \leq x_{ijsk}$$

$$is\_apprenti_{ijsk} \leq z_{ijs}^1$$

$$is\_apprenti_{ijsk} \leq (1 - has\_level_{ijsk})$$

$$is\_apprenti_{ijsk} \geq x_{ijsk} + z_{ijs}^1 - has\_level_{ijsk} - 1$$

$$tab\_count\_tasks\_has\_apprenti_k^m + = is\_apprenti_{ijsk}$$

## Linéarisation du ET LOGIQUE

tab\_count [k][m] = nombre de fois  
où l'opérateur k à été apprenti pour  
une tache de corps de métier m

# Savoir si un worker a fait la tache en collaboration

$$\forall i \in J, j \in J_i, s \in O_{ij}, k \in W, r = O_{ijs}, r \in m$$

$$is\_collab_{ijsk} \leq x_{ijsk}$$

$$is\_collab_{ijsk} \leq z_{ijs}^2$$

$$is\_collab_{ijsk} \geq x_{ijsk} + z_{ijs}^2 - 1$$

$$tab\_count\_tasks\_with\_collab_k^m + = is\_collab_{ijsk}$$

## Linéarisation du ET LOGIQUE

tab\_count [k][m] = nombre de fois  
où l'opérateur k à été en collab pour  
une tache de corps de métier m

**Faire augmenter charge cognitifs selon le nombre de tache réalisé selon les modes puis multiplier par un COEFF dépendant du mode**

$$cognitive\_load\_tutors_k^m = tab\_count\_tasks\_has\_tutor_k^m \cdot T$$

$$cognitive\_load\_appr_k^m = tab\_count\_tasks\_has\_apprenti_k^m \cdot A$$

$$cognitive\_load\_collab_k^m = tab\_count\_tasks\_with\_collab_k^m \cdot C$$

**Agrégation des charges cognitifs pour un worker k selon le métier m**

$$\begin{aligned} cognitive\_load\_total_k^m = & \\ & cognitive\_load\_tutors_k^m \\ & + cognitive\_load\_apprentis_k^m \\ & + cognitive\_load\_collab_k^m \end{aligned}$$

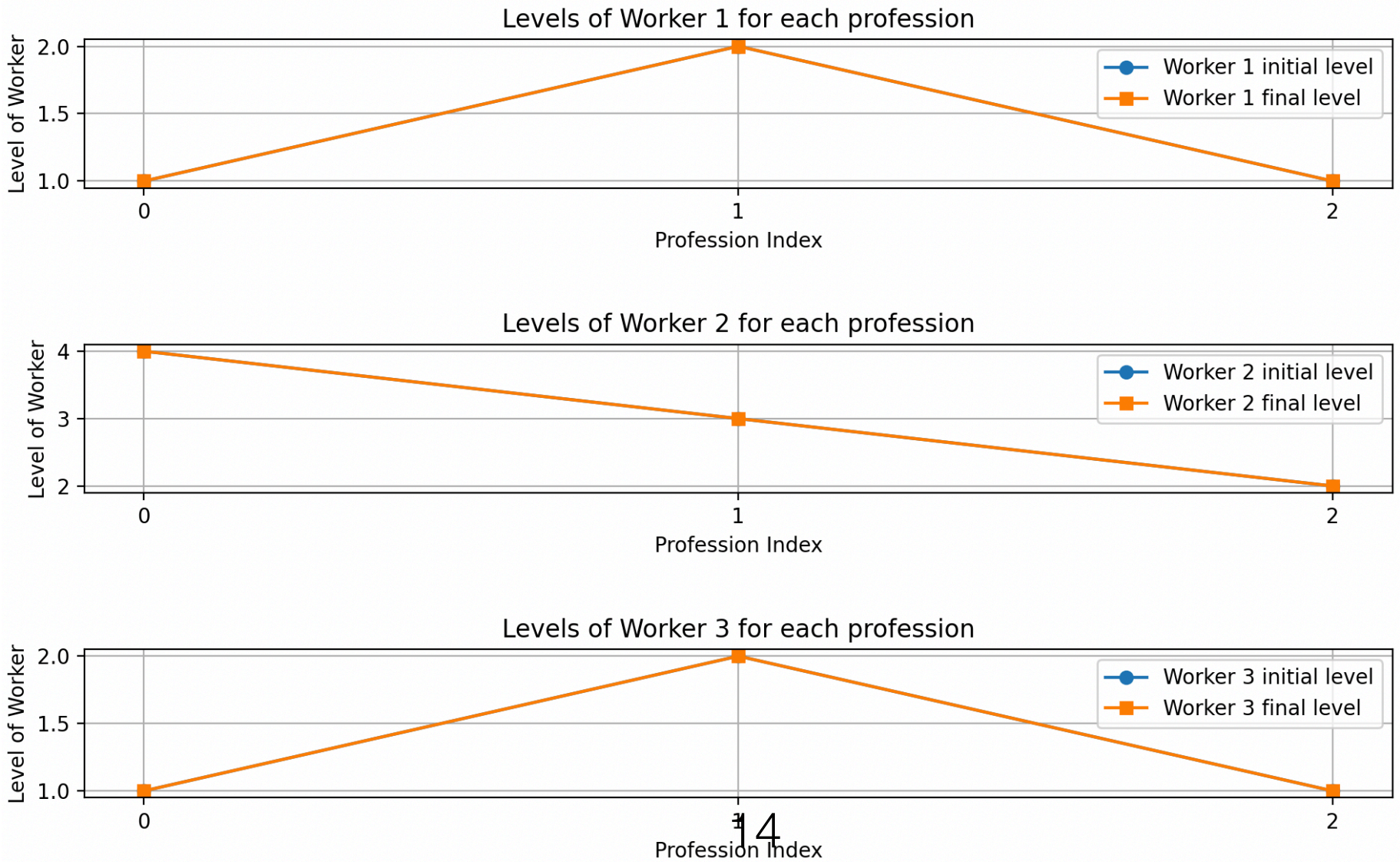
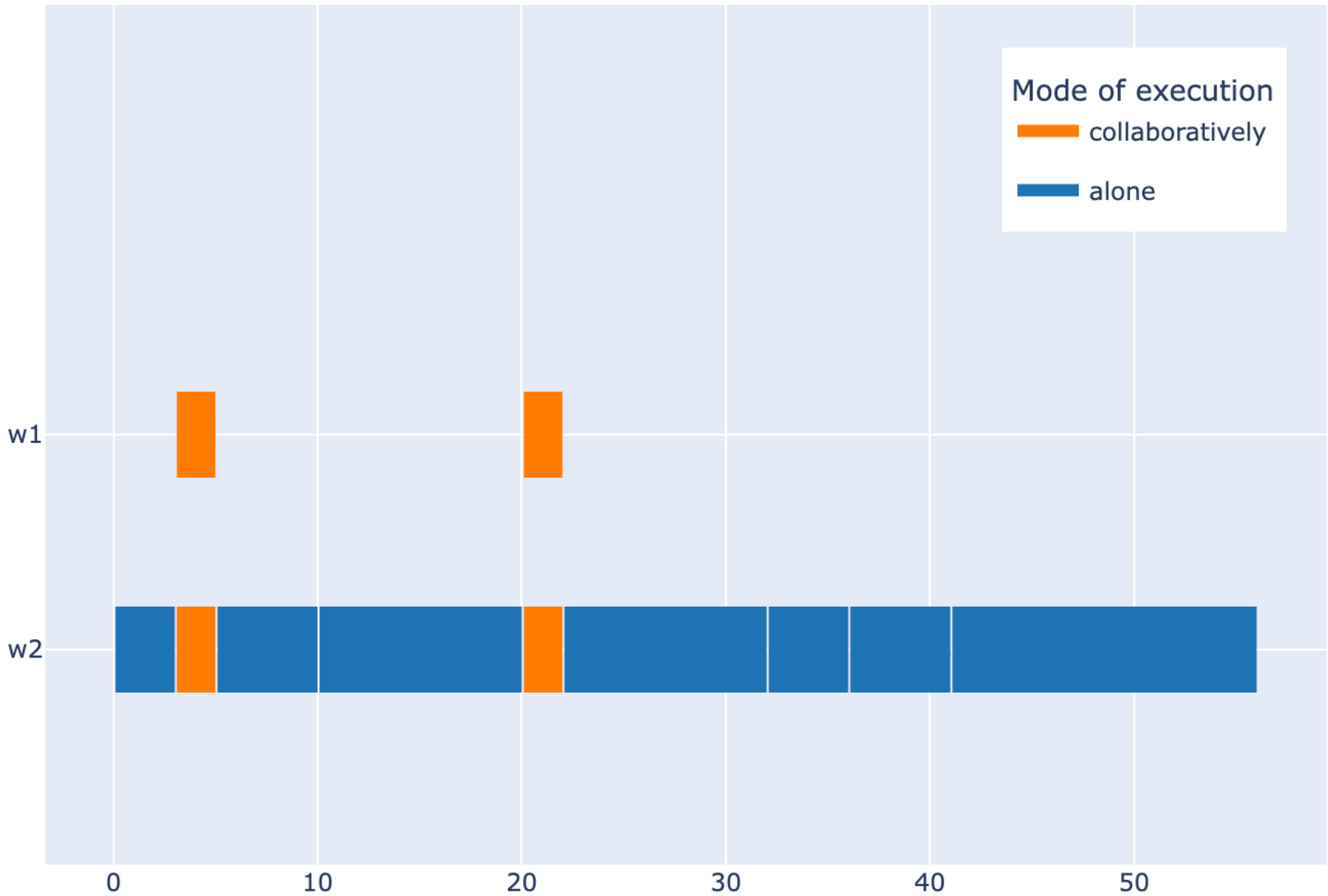
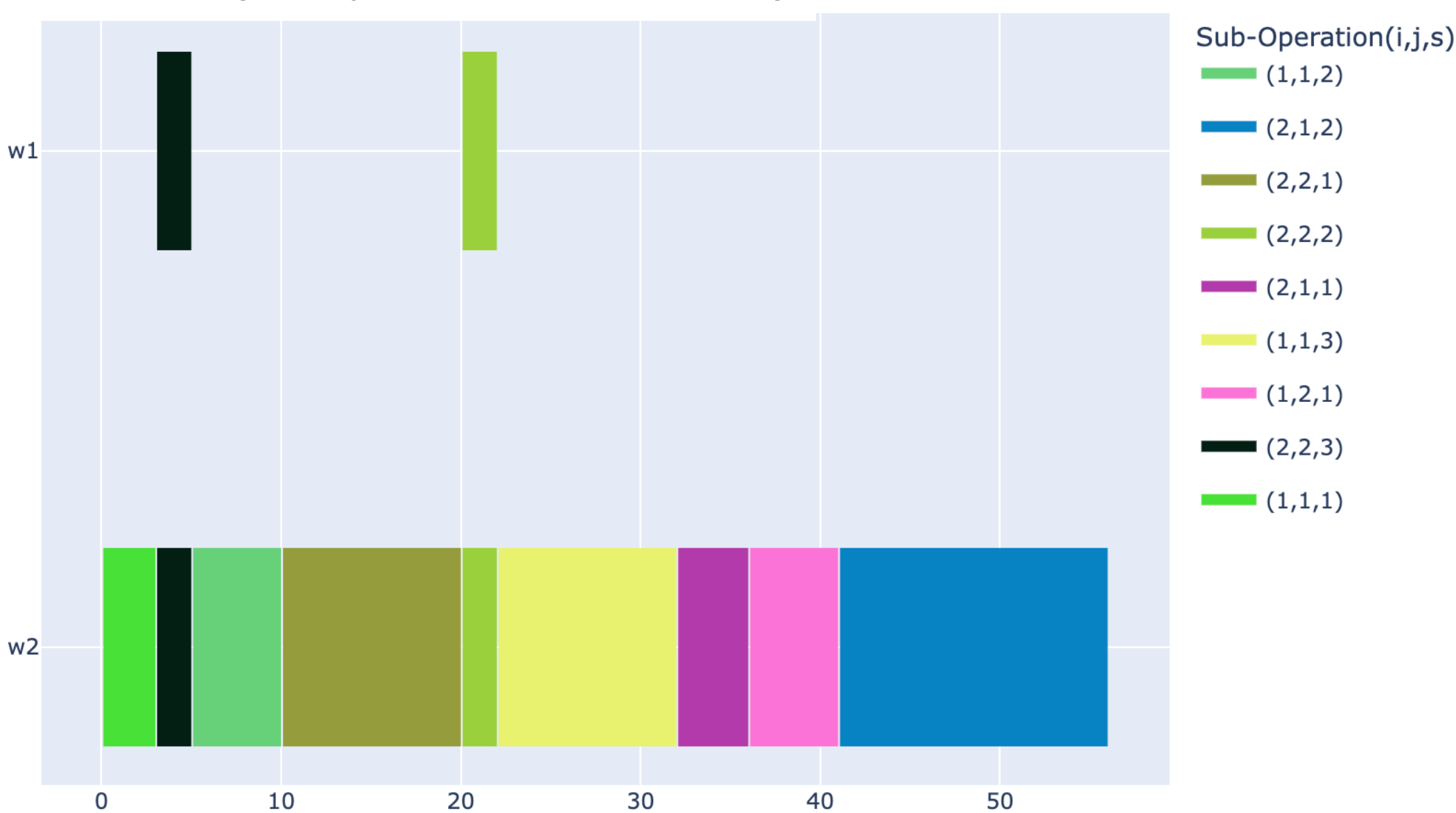
# **Quelques Visualisations sur le notebook avec prise en compte de la charge cognitif**

# Quelques Résultats

# Min makespan

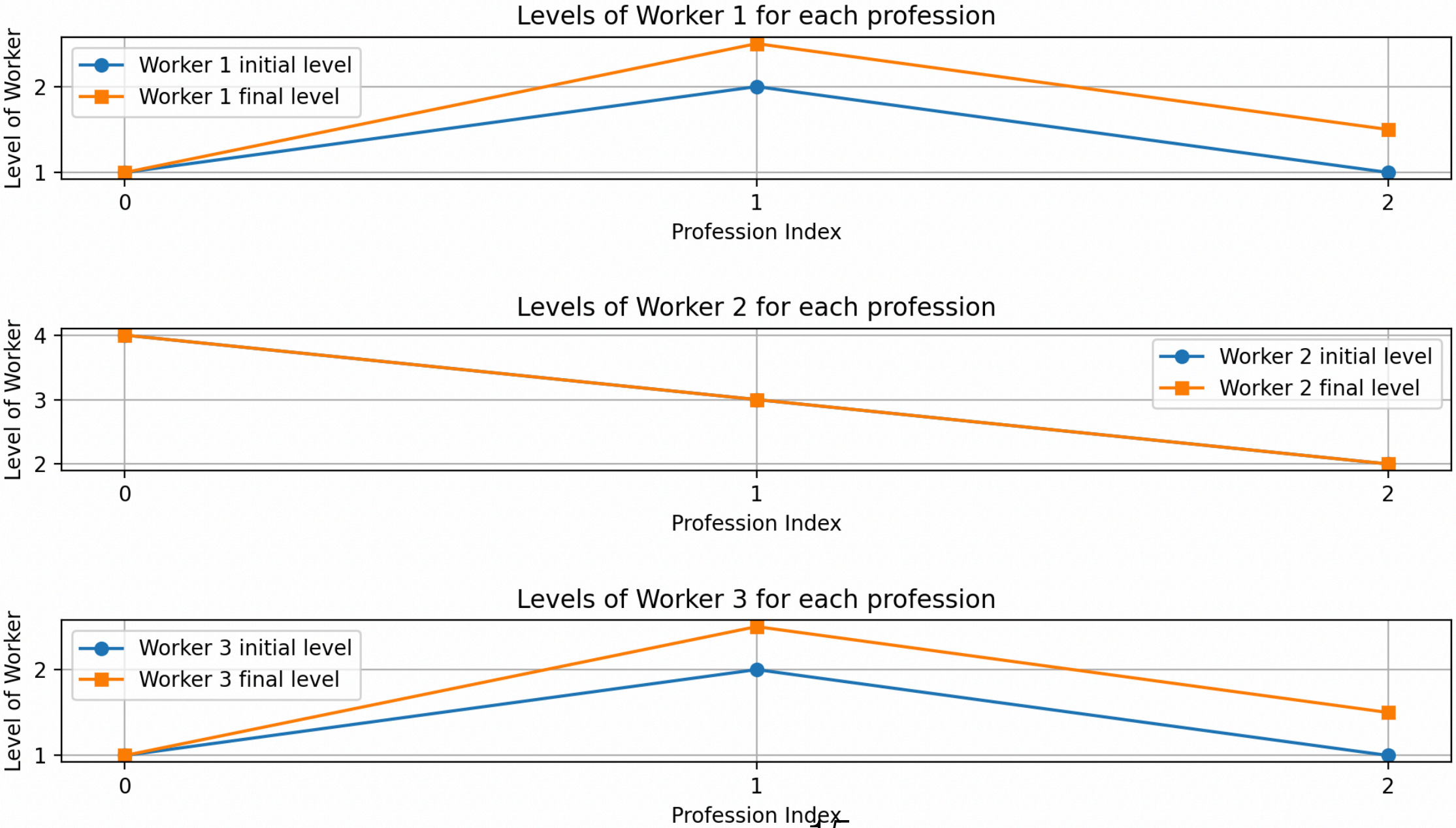
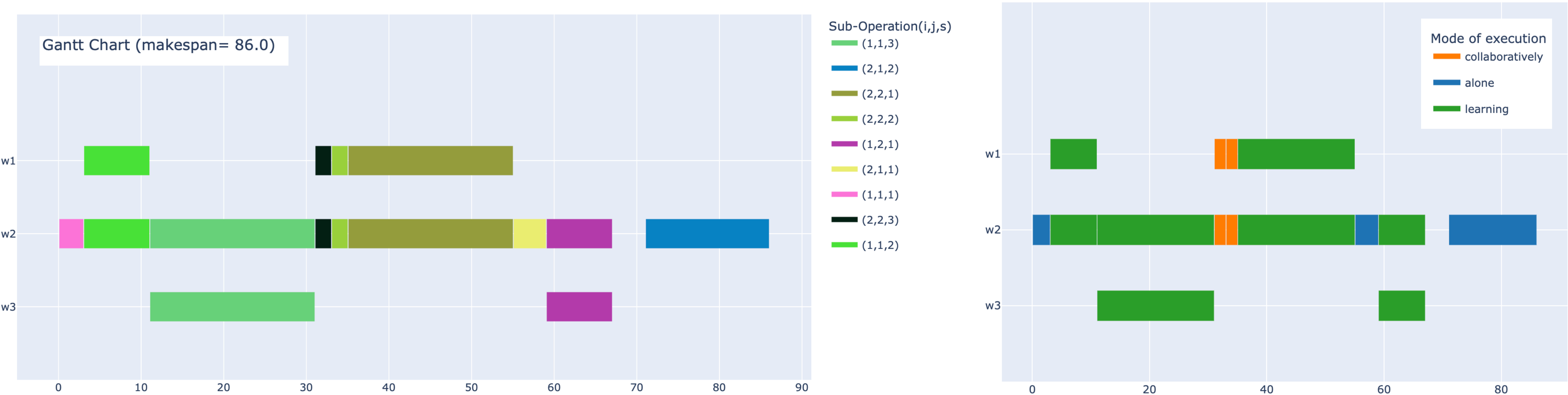
**\*Je n'avais pas encore pris en compte les charges cognitifs**

Gantt Chart (makespan= 55.99999998831546)



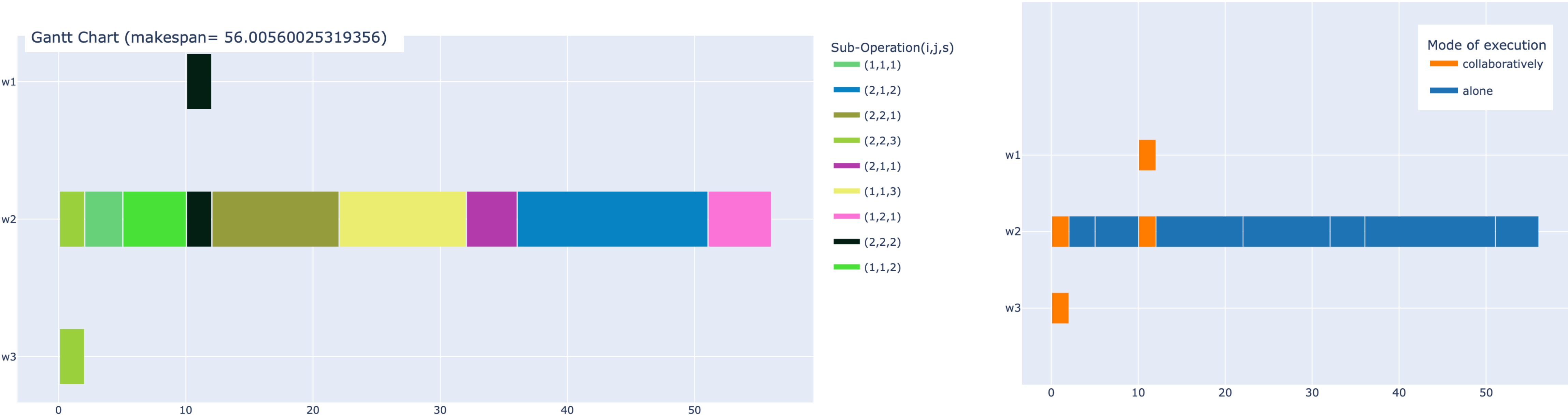
Max skill

**\*Je n'avais pas encore pris en compte les charges cognitifs**

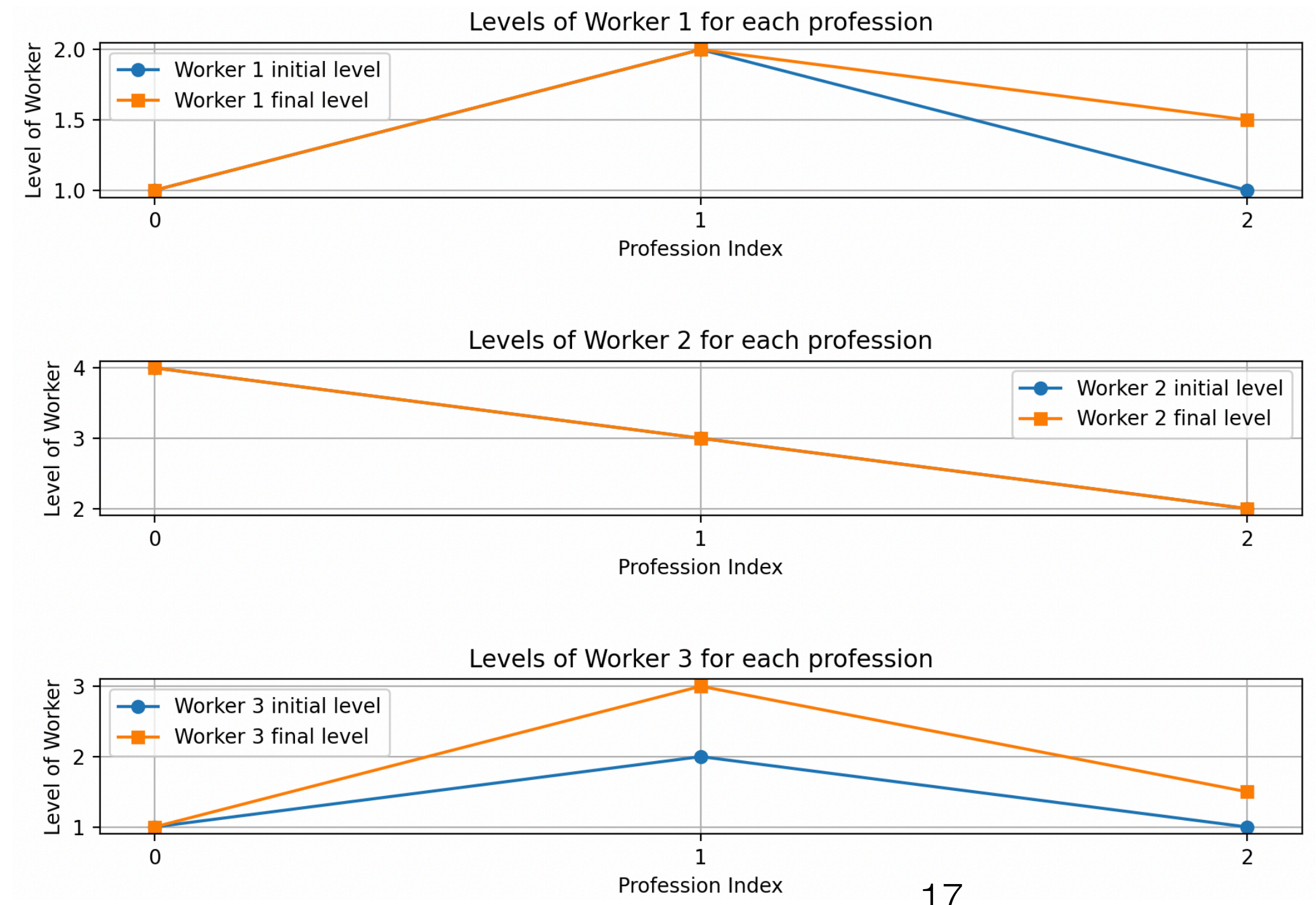
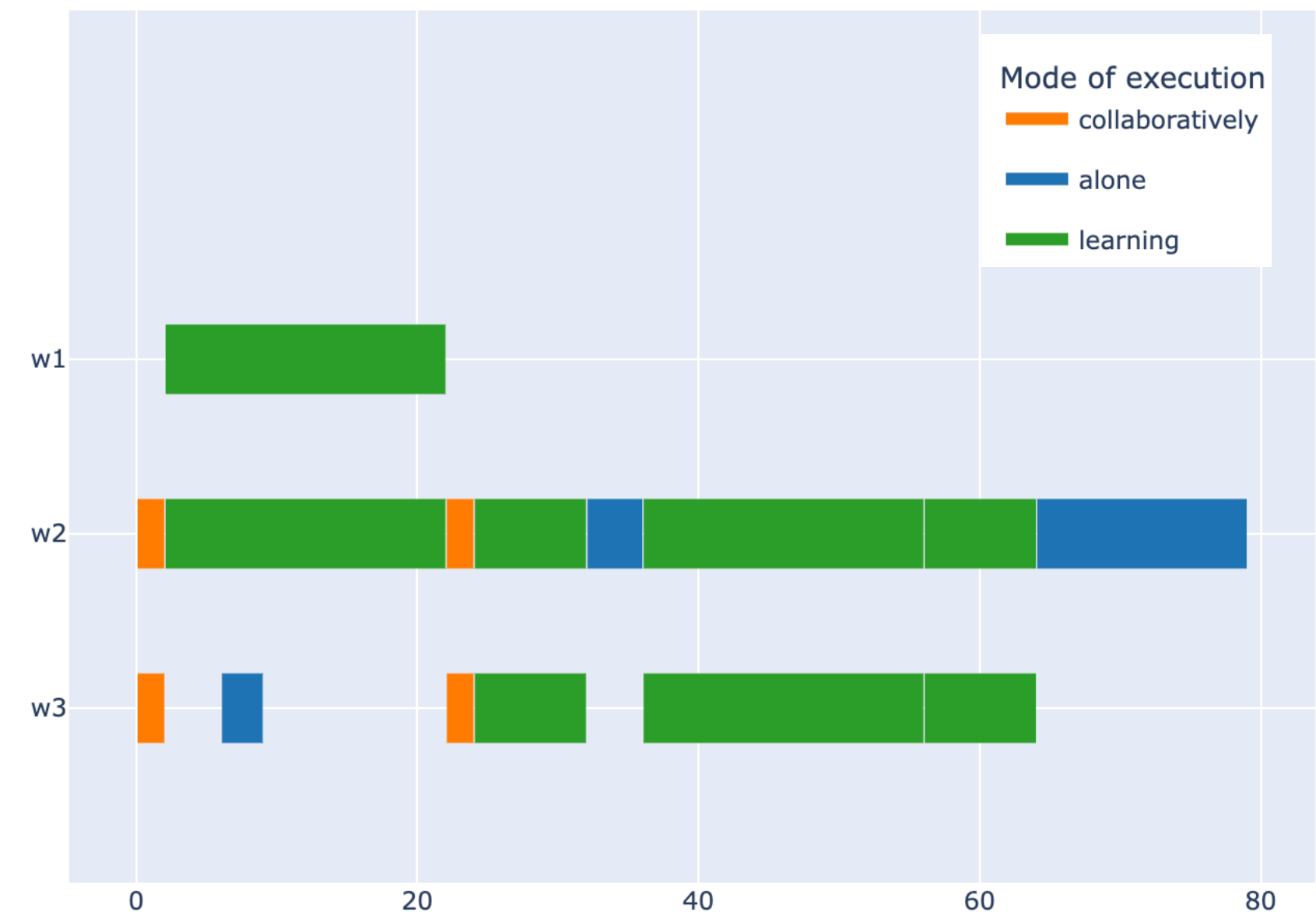
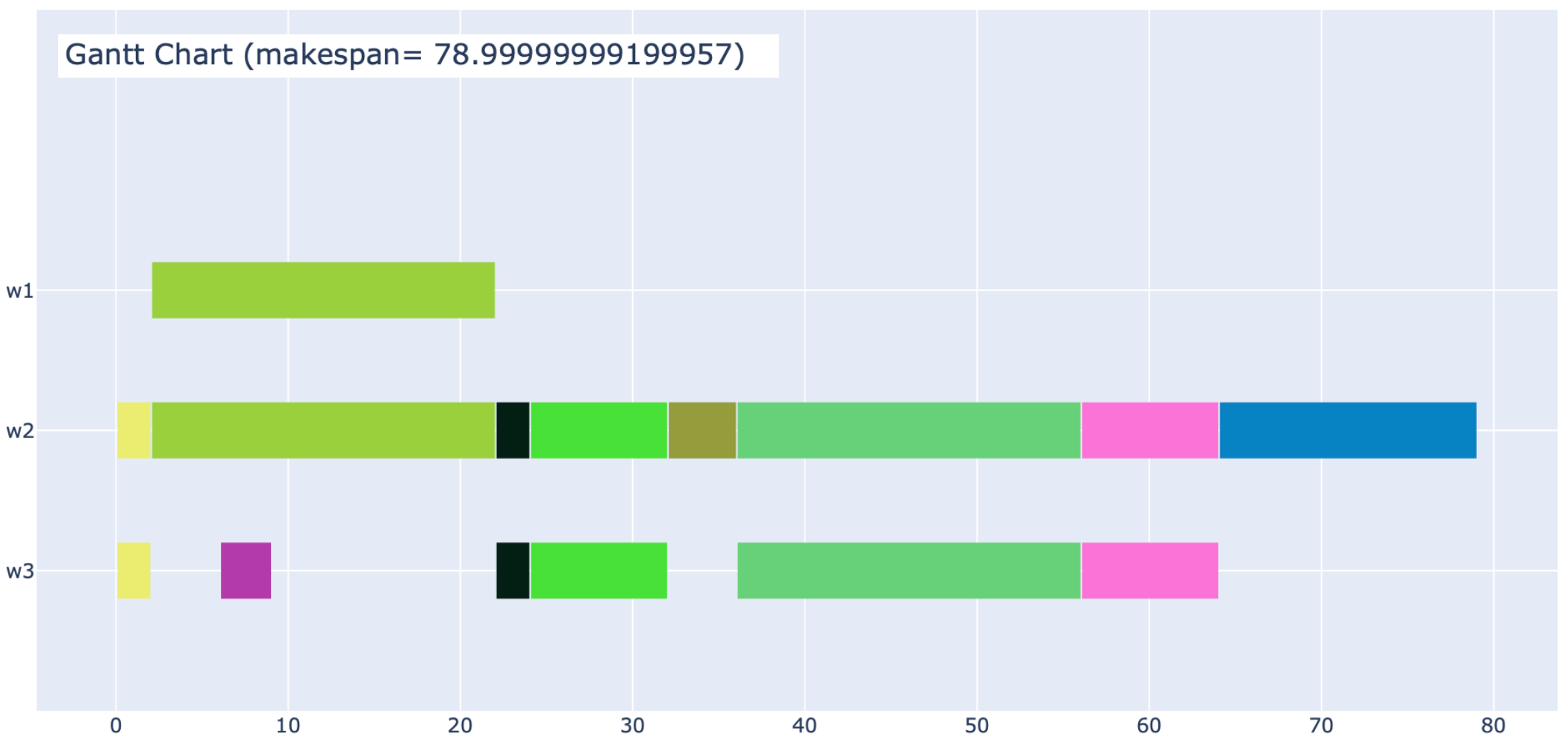


**\*Je n'avais pas encore pris en compte les charges cognitifs**

**LEXICOGRAPHIQUE**    Min makespan, puis Max skill



# LEXICOGRAPHIQUE    Max skill, puis Min makespan



**\*Je n'avais pas encore pris en compte les charges cognitifs**

# Objectifs du modèle

Pour le moment:

- Max somme des skills
- Min somme des charges cognitifs

Ne donne pas des solutions équitables à priori (supposition pour la suite)

- Max Min skills
- Min Max charges cognitifs
- Ou autre ...

## 1. Min Makespan

- Finir les jobs rapidement

## 2. Max Skill

- Former des opérateurs pour qu'ils puissent avoir plus d'impact par la suite

## 3. Min charge cognitif des tuteurs

- Lors de la formation d'opérateur, la charge cognitive des tuteurs augmente

## 4. Min charge cognitive des apprentis

- Lors de la formation des apprentis, leur charge cognitive augmente

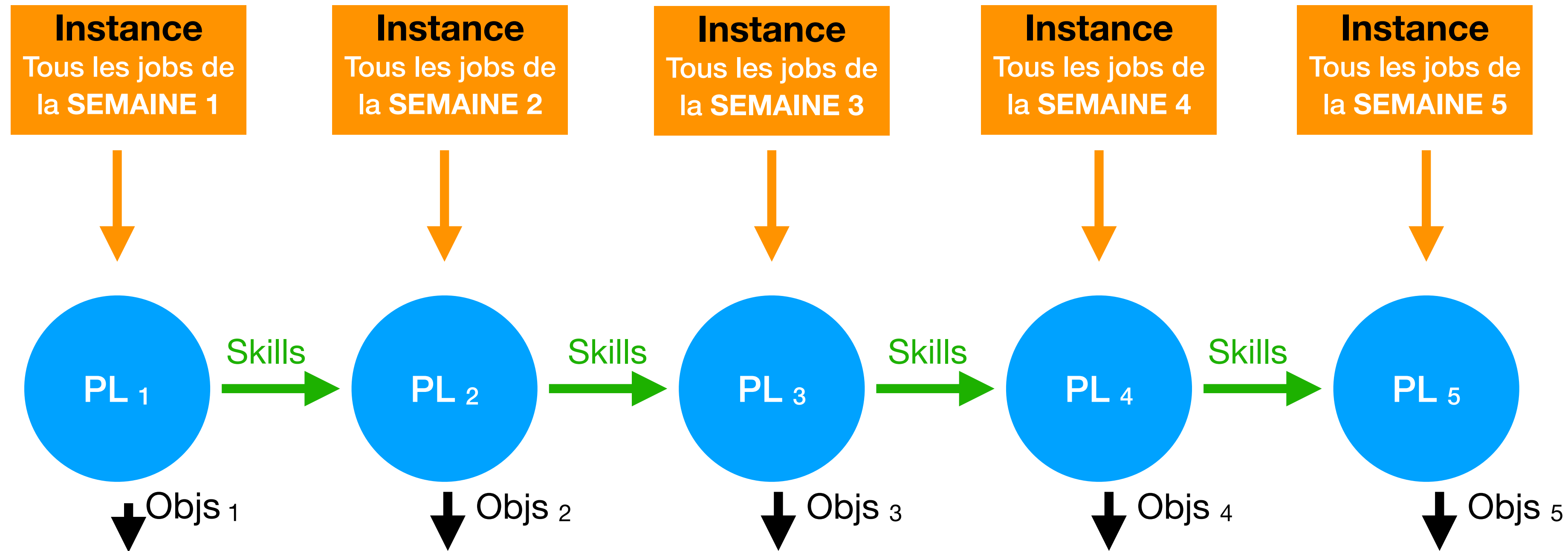
## 5. Min charge cognitive des tâches réalisées en collaboration

- Lorsqu'une tâche est exécutée en collaboration, la charge cognitive des opérateurs augmente

**Actuellement j'ai sommé les points 3, 4 et 5 pour ne former qu'un seul objectif.  
Sinon mettre une borne sup à ne pas dépasser, pour être en bi-objectifs**

# Planification Multi-Période (ex : semaine)

Une instance par PL



Hypothèses / Questionnements :

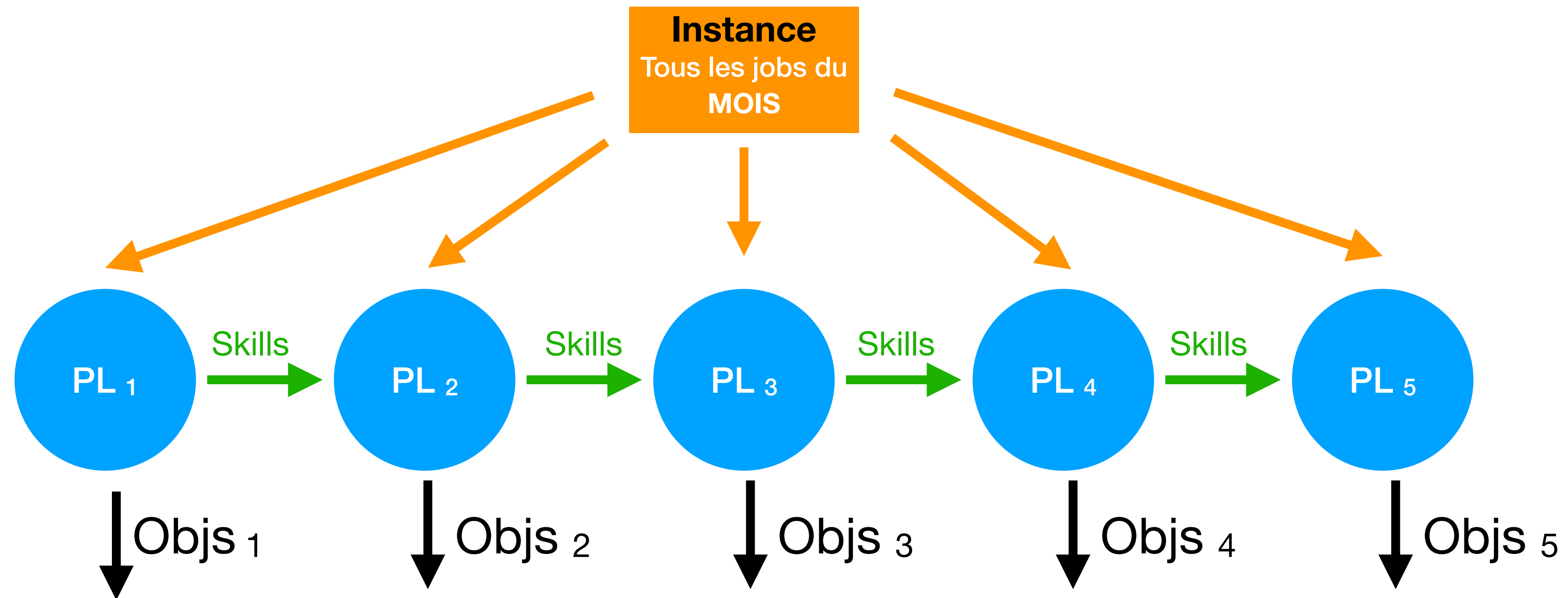
- L'instance prend en compte les tâches à faire pour la semaine en question seulement ?
- Tous les jobs de l'instance  $i$  doivent être exécutés dans la semaine  $i$  **OU** les jobs non terminés peuvent continuer dans les autres semaines (  $PL_{i-1}$  doit donner info à  $PL_i$  )
- Chaque  $PL_i$  résout les affectations de la semaine  $i$
- Ne pas faire travailler les opérateurs plus de  $x$  heures par jours et plus de  $y$  heures chaque semaine
- L'instance d'une semaine à l'autre change (démissions opérateurs, nouveaux opérateurs, skills modifiés selon PL des semaines précédentes)

# Planification Multi-Période (ex : semaine)

Une instance pour tout les PL

Hypothèse :

- L'instance ne change pas durant la semaine



Tous les jobs disponible (de l'instance) qui n'ont pas pu être terminé dans une semaine, peuvent reprendre aux semaines d'après

Idée / Questionnement :

- Avoir ou non une date Limite par jobs

# Planification Multi-Période (ex : semaine)

Optim même fonction obj

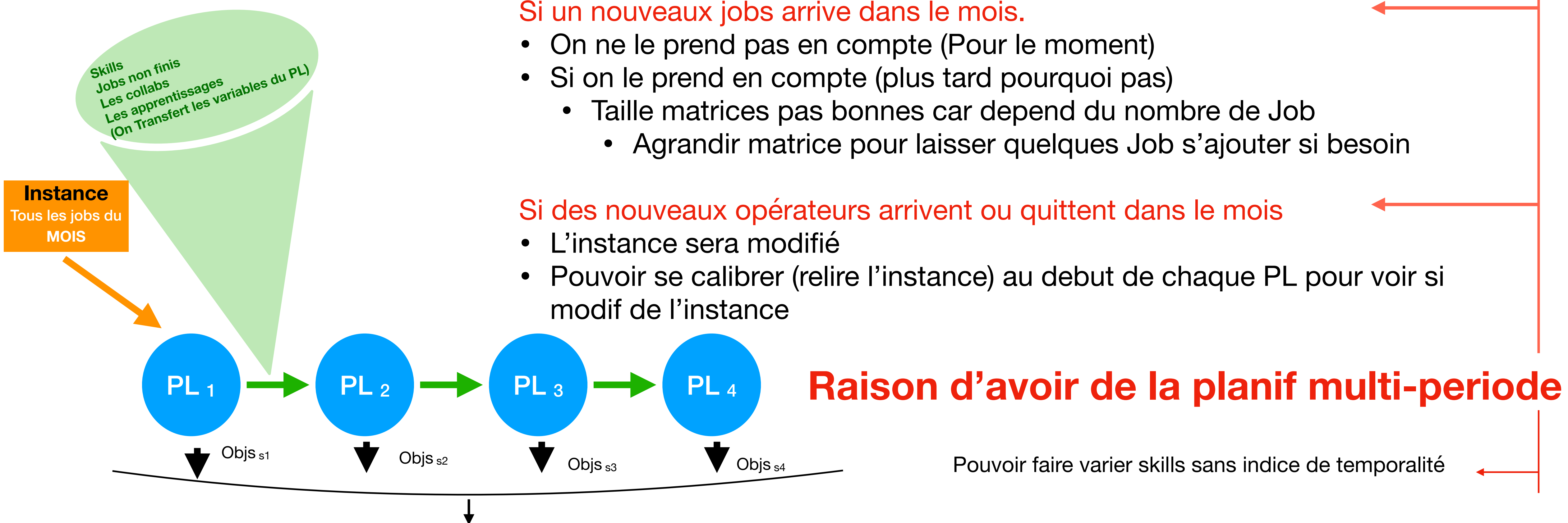


Optim **skills** dans une  
periode puis **makespan**  
ou l'inverse (tout en min  
charge cognitif)



Optim même fonction obj





**Si un nouveaux jobs arrive dans le mois.**

- On ne le prend pas en compte (Pour le moment)
- Si on le prend en compte (plus tard pourquoi pas)
  - Taille matrices pas bonnes car depend du nombre de Job
  - Agrandir matrice pour laisser quelques Job s'ajouter si besoin

**Si des nouveaux opérateurs arrivent ou quittent dans le mois**

- L'instance sera modifié
- Pouvoir se calibrer (relire l'instance) au debut de chaque PL pour voir si modif de l'instance

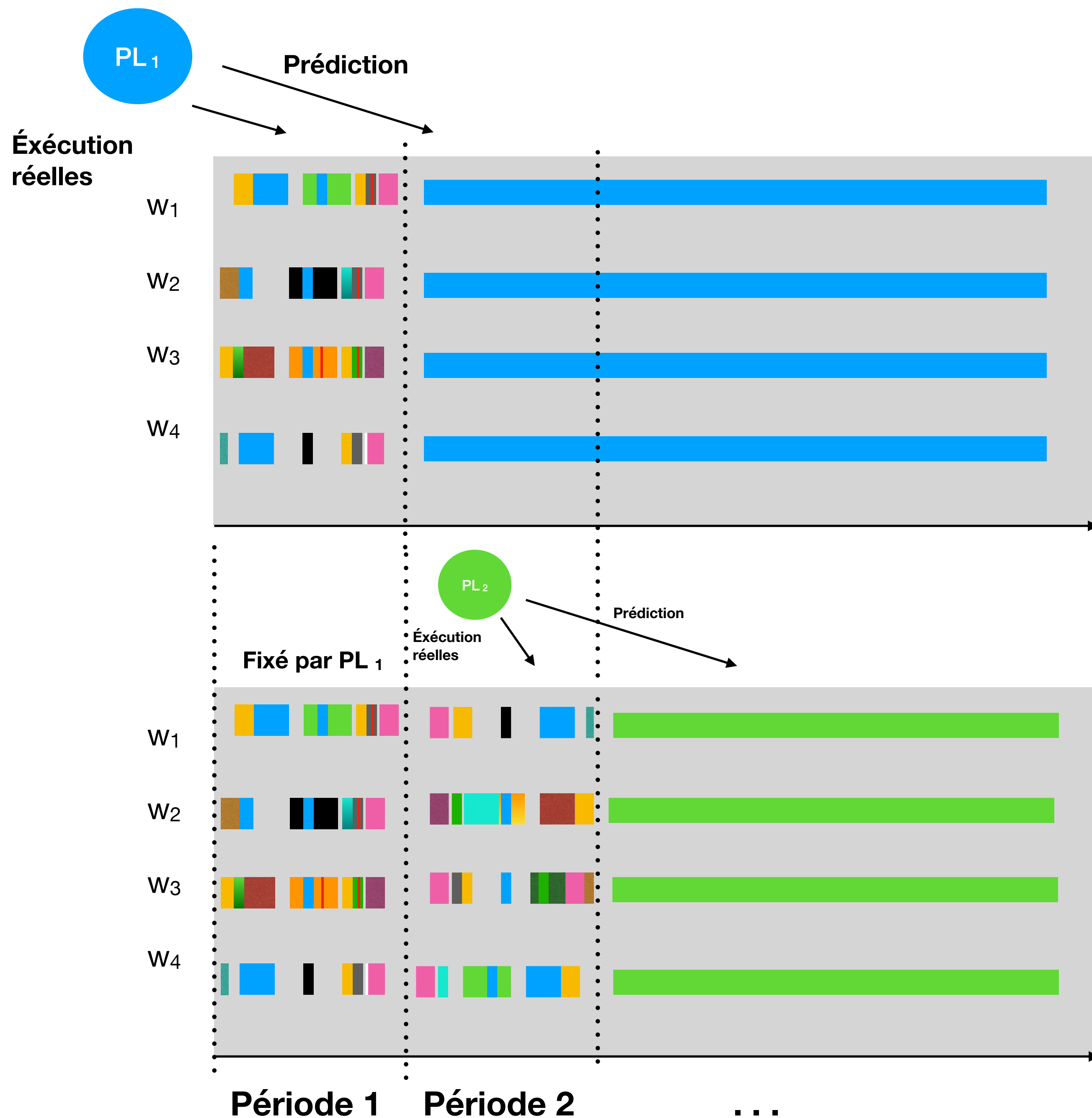
1. ☒ L'objectif trouvé en semaine i est l'objectif cumulé des semaine précédente (les charge cognitif se répercute d'une semaine à l'autre)
  - L'objectifs de la semaine 4 est celle du mois
  - Post traitement regarder via variables du PL regarder les stats (skills, jobs done, cognitive load) pour chaque semaine, chaque jours, etc..
2. ☐ L'objectif est calculé pour chaque semaine indépendamment

- Avoir une limite de makespan local à la semaine
- Le PL doit prendre en compte les dates de début de chaque affectations
- Ex: le PL1 Donne des affectations à des dates de 0 jusqu'à la limite du makespan local
  - Puis en post-traitement on récupère les variables du PL pour mapper avec des dates et heures précise de la semaine (9h/12h - 14h/18h)
  - Ou on contraint les variables de debut et fin du PL pour les calibrer sur ces horaires (Je vais essayer cela dans un premier temps)

**Après avoir Réfléchis.**

**Peut être faire un PL par jours pour ne pas avoir d'indice de temporalité sur nos variables ...**

**A voir (zone assez flou pour le moment)**



**Chaque PL<sub>i</sub> doit connaître le temps de chaque période.**

**Pour le prendre en compte dans leurs prédiction même si obsolète car les skills peuvent évoluer**

**Le PL<sub>i</sub> doit garder fixes les taches exécuté pendant la période i-1**

Questionnement :

Si on considère que un PL donne l'affectation pour une semaine  
 -> le temps de traitement des taches doit être proportionelle

Au moins 1 personnes Ayant un niveau de compétence requis pour l'exécution d'une tâche

Hard constraints

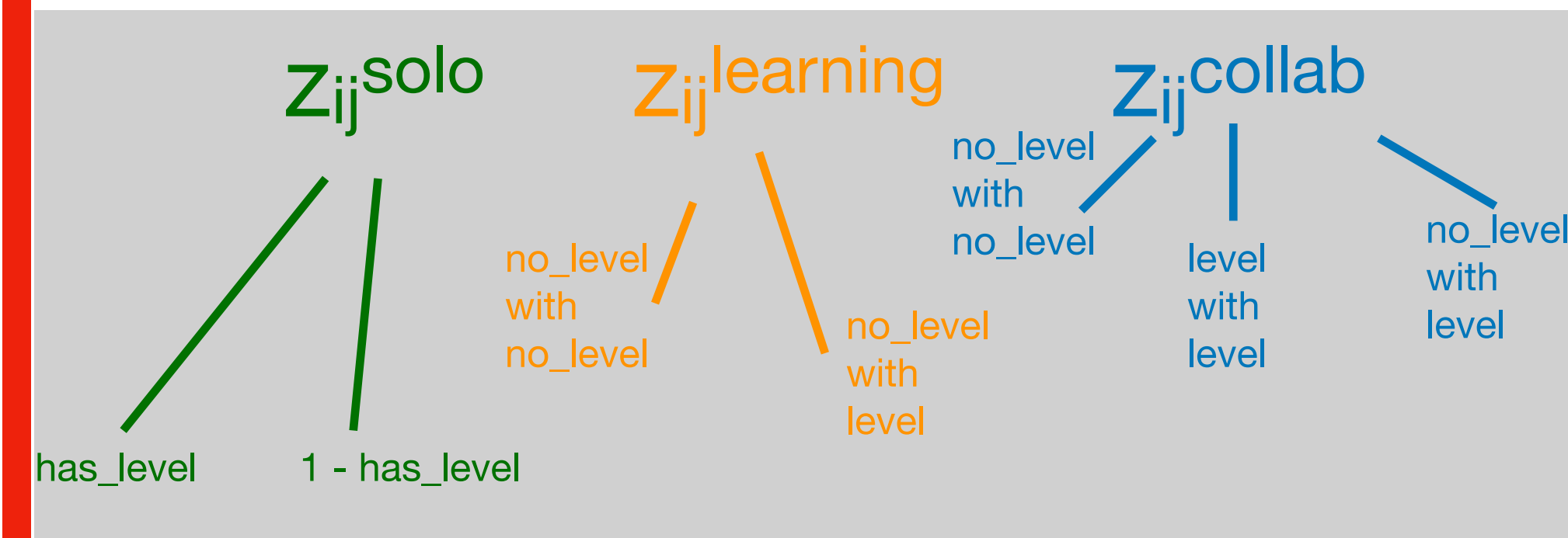
$\forall i \in J, j \in J_i, j' \in J_i, s \in O_{ij}, k \in W, L_m^k < d_r$

$$x_{ijsk} + \sum_{k' \in W, k' \neq k} x_{ijsk'} \geq M \cdot (x_{ijsk} - 1) + 2$$

$$\sum_{k' \in W, k' \neq k} x_{ijsk'} \cdot L_m^{k'} \geq d_r \cdot x_{ijsk}$$

Worker k n'a pas le niveau requis pour la tâche O<sub>ijs</sub>

- S'il est affecté alors une autre personne doit être affecté avec lui
- Sinon contrainte non actif (big M)
- Si k fait la tâche alors la personne avec lui doit forcément avoir le niveau de compétence requis
- Sinon contrainte non actif



Soft constraints

2

Si aucun worker compétent pour O<sub>ij</sub> alors on l'affecte au worker qui à la qualification la plus proche

$$x_{ijk} + \sum_{k' \in W, k' \neq k} x_{ijk'} \geq M \cdot (x_{ijk} - 1) + 2 - \text{penalty\_level} \cdot L_{km}$$

penalty\_level : global  $\in \mathbb{R}_+$   
penalty\_level<sub>ij</sub> : spécifique à chaque tâche  
On souhaite **min** penalty\_level

$$\sum_{k' \in W, k' \neq k} x_{ijk'} \cdot L_{km} \geq D_{ij} \cdot x_{ijk} - M (\text{penalty\_level})$$

Si x<sub>ijk</sub> = 1 n'a pas le level et trouve une personne compétente :  
c1: 2 >= 2 et c2: 3 >= 3 -> avec penalty\_level = 0 et on est content

Si x<sub>ijk</sub> = 1 n'a pas le level et personne dans l'usine possède le level requis on a :  
c1: 1 >= 2 - penalty\_level \* L<sub>km</sub> et c2: 0 >= D<sub>ij</sub> - M(penalty\_level)

(c1): 1 = penalty \* L<sub>km</sub>  
(1/L<sub>km</sub>) = penalty  
L<sub>km</sub> <= 4 donc (1/4) <= penalty  
(c2) => 0 >= 4 - M/4  
Donc **M=16**

Tâche de niveau 4 et on a 3 personnes de level suivant l'ordre 1, 2 et 3

Entre : penalty\_level \* 1, penalty\_level \* 2, penalty\_level \* 3 -> il va se caler sur le worker de niveau 3  
=1                      =1/2                      =1/3

Donc maintenant on autorise des tâches à être fait seul sans avoir la compétence

$$\forall i \in J, j \in J_i$$

OLD

$$\text{c1} \quad z_{ij}^{solo} + z_{ij}^{learn} + z_{ij}^{collab} = 1$$

$$\text{c2} \quad \sum_{k \in W} x_{ijk} = 1 \cdot z_{ij}^{solo} + 2 \cdot z_{ij}^{learn} + 2 \cdot z_{ij}^{collab}$$

$$\forall i \in J, j \in J_i, s \in O_{ij}, r = O_{ijs}$$

(12)

OLD

$$\text{c3} \quad LMIN_{ijs} + M \cdot z_{ijs}^1 \geq d_r$$

$$\text{c4} \quad LMIN_{ijs} - M \cdot z_{ijs}^2 \leq d_r + M \cdot \left( 2 - \sum_{k \in W} x_{ijsk} \right)$$

solo :  $z_{ijs}^1$  non contraint

appr :  $z_{ijs}^1 = 1$

collab :  $z_{ijs}^1$  non contraint

solo :  $z_{ijs}^2$  non contraint

appr :  $z_{ijs}^2$  non contraint

collab :  $z_{ijs}^2 = 1$

Plus valable car considèrerait

- Que si level\_min < Difficulté\_tache alors apprentissage
- Que tache en solo implique que k a le niveau requis

$$\forall i \in J, j \in J_i$$

c1<sup>new</sup>

$$z_{ij}^{solo\_level} + z_{ij}^{solo\_no\_level} + z_{ij}^{learn} + z_{ij}^{collab} = 1$$

c2<sup>new</sup>

$$\sum_{k \in W} x_{ijk} = 1 \cdot z_{ij}^{solo\_level} + 1 \cdot z_{ij}^{solo\_no\_level} + 2 \cdot z_{ij}^{learn} + 2 \cdot z_{ij}^{collab}$$

c2.1<sup>new</sup>

$$L_{km} \geq D_{ij} \cdot x_{ijk} - M_4 \cdot z_{ij}^{solo\_no\_level} - M_4 \cdot z_{ij}^{learn}$$

c2.2<sup>new</sup>

$$L_{km} \leq (D_{ij} - \epsilon) \cdot x_{ijk} + (1 - x_{ijk}) \cdot M_4 + M_4 \cdot z_{ij}^{solo\_level} + M_4 \cdot z_{ij}^{learn} + M_4 \cdot z_{ij}^{collab}$$

$\forall i \in J, j \in J_i, k \in W, O_{ij} \in m$

Sert si solo SANS level, mettre solo\_level = 0

Sert si solo AVEC level, mettre solo\_level = 1

Si x ne fait pas , pas contraint

Si x fait et a le niveau -> learn ou collab

Si x fait sans niveau -> learn ou solo SANS level

$$\forall i \in J, j \in J_i$$

c3<sup>new</sup>

$$level\_min_{ij} + M_4 \cdot z_{ij}^{learn} + M_4 \cdot z_{ij}^{solo\_no\_level} \geq D_{ij}$$

c4<sup>same</sup>

$$level\_min_{ij} - M \cdot z_{ij}^{collab} \leq D_{ij} - \epsilon + M \cdot \left( 2 - \sum_{k \in W} x_{ijk} \right)$$

+ 20% du temps si fait en solo sans niveau

$$d_{ijk} + T_{ij}^{solo} \cdot z_{ij}^{solo\_level} + (T_{ij}^{solo} \cdot z_{ij}^{solo\_no\_level} \cdot 1.2) + T_{ij}^{learn} \cdot z_{ij}^{learn} + T_{ij}^{collab} \cdot z_{ij}^{collab} - M(1 - x_{ijk})$$

26

Number of jobs: 4  
Number of professions: 3  
Number of tasks per profession: [2 2 3]  
Max number of operations per Jobs: 7  
Total number of tasks: 7  
Total number of workers: 7  
Task to profession mapping:  
{0: 0, 1: 0, 2: 1, 3: 1, 4: 2, 5: 2, 6: 2}  
Worker levels: shape=(7, 3)  
[[2 2 2]  
[1 2 1]  
[2 4 4]  
[3 4 3]  
[4 1 4]  
[3 3 2]  
[1 4 2]]  
Job difficulties: shape= (4,)  
[2 4 2 4]  
Task difficulties: shape= (7,)  
[1 1 3 2 4 1 4]  
Task processing times: shape= (7, 3)  
[[ 7. 8.44322744 5.33664751]  
[19. 28.32848616 13.60347768]  
[11. 15.94706071 7.10176027]  
[11. 13.90071907 8.86519092]  
[ 4. 5.01818996 2.27898772]  
[ 8. 10.04017082 -1. ]  
[ 3. 3.87381802 2.04954276]]  
Jobs structure: len= 4  
Job 0 : 0\_(0,0) 0\_(0,1) 0\_(0,2) 0\_(0,3) 0\_(0,4) 0\_(0,5)  
Job 1 : 0\_(1,0) 0\_(1,1) 0\_(1,2) 0\_(1,3) 0\_(1,4) 0\_(1,5)  
Job 2 : 0\_(2,0) 0\_(2,1) 0\_(2,2) 0\_(2,3)  
Job 3 : 0\_(3,0) 0\_(3,1) 0\_(3,2) 0\_(3,3) 0\_(3,4)

seed=42

